

Assembly for Malware Analysis

4/15/26 2:31 PM

1. CPU Architecture & Number Systems

CPU basics (what malware interacts with):

- **Registers:** Very fast storage inside the CPU. Malware constantly moves data in and out of registers.
 - Common 32-bit: EAX, EBX, ECX, EDX, ESI, EDI, EBP, ESP
 - 64-bit version: RAX, RBX, etc.
 - EAX/RAX often holds return values from functions.
 - ESP/RSP = stack pointer (points to top of stack).
 - EBP/RBP = base pointer (helps access local variables and parameters).

Example in assembly (what you see in IDA Pro):

```
mov eax, 5 ; put 5 into EAX register
mov ecx, [ebp+8] ; read parameter from stack into ECX
```

Memory: RAM where variables, code, and the stack live. Malware reads and writes memory constantly.

Stack: Special area of memory used for function calls, local variables, and saving return addresses. It grows downward (lower addresses).

Number systems:

- **Hex** (base 16): Most common in disassembly. Every byte = two hex digits.
- **Why hex?** Easy to read raw bytes and addresses.

Example:

```
0x4D5A ; "MZ" - start of every Windows executable (PE header)
```

Malware tip: When you see 0x in IDA, it is hex. You will use hex for addresses, opcodes, and constants.

2. Basic Assembly Instructions

Assembly instructions tell the CPU what to do. In malware analysis you read these in IDA Pro to understand logic.

Most common instructions (memorize these):

- MOV dest, src → copy value

```
mov eax, 10 ; EAX = 10
mov ebx, eax ; EBX = EAX
```

- PUSH value → put value on stack (stack grows down)

```
push 5 ; put 5 on stack
```

- POP dest → take value from stack

```
pop eax ; take value from stack into EAX
```

- CALL function → call a function (pushes return address)

```
call sub_401000 ; call function at address 401000
```

- RET → return from function

```
ret ; pop return address and jump back
```

- ADD / SUB → math

```
add eax, 5 ; EAX = EAX + 5
sub esp, 0x20 ; allocate 32 bytes on stack for locals
```

- CMP op1, op2 → compare (sets flags)

```
cmp eax, 5 ; compare EAX with 5
```

- JMP label → unconditional jump

```
jmp end_loop ; always jump to end_loop
```

- Conditional jumps (Jcc):

```
je equal_label ; jump if equal (from CMP)
jne not_equal ; jump if not equal
```

- jg greater ; jump if greater
- jl less ; jump if less

- Malware tip: In IDA Pro, blue arrows = jumps, red arrows = calls. Follow them to understand control flow.

3. Functions & The Stack

Stack basics:

- Used for local variables, function parameters, and return addresses.
- ESP/RSP always points to the top.

Typical function structure (what you see in every malware function):

```
sub_401000: ; function start
push ebp ; save old base pointer
mov ebp, esp ; set new base pointer
sub esp, 0x20 ; allocate space for local variables

; function body here

mov esp, ebp ; clean up stack
pop ebp ; restore old base pointer
ret ; return
```

Calling conventions (how arguments are passed):

- **cdecl** (common in 32-bit malware): Arguments pushed on stack right-to-left. Caller cleans stack (add esp, X).
- **x64** (64-bit): First 4 arguments in registers (RCX, RDX, R8, R9), rest on stack.

Malware tip: Look for push ebp; mov ebp, esp — this marks the start of a function in IDA Pro.

4. Control Flow (Ifs, Loops, Switches)

If statements:

```
cmp eax, 5 ; compare
jne else_part ; if not equal, jump to else
; if true code here
jmp end_if
else_part:
; else code here
end_if:
```

Loops (for / while):

```
mov ecx, 0 ; i = 0
loop_start:
cmp ecx, 10
jge loop_end ; if i >= 10, exit

; loop body

inc ecx ; i++
jmp loop_start
loop_end:
```

Switches: Often use a jump table (array of addresses). Malware uses switches for command handling or anti-analysis.

Malware tip: In IDA Pro, look for CMP followed by conditional jump (Jcc).

Upward jumps = loops. Downward jumps = if/else.

5. Arrays, Structures & Pointers

Pointers: A value that holds a memory address.

Example:

```
lea eax, [string] ; load address of string into EAX (pointer)
mov ebx, [eax] ; read value at that address
```

Arrays: Contiguous memory. Accessed with offset.

Example:

```
mov eax, [commands + ecx*4] ; read 4th element (ecx=3) of array
```

Structures: Group of variables. Accessed with fixed offsets.

Example (Config struct):

```
mov eax, [ebp-0x10] ; cfg.port (offset 0x10)
mov ebx, [ebp-0x0C] ; cfg.url (offset 0x0C)
```

Malware tip: In IDA Pro, press T on a register to apply a structure. Repeated offsets (+4, +8, +0xC) often mean array or struct.

6. Complete Example: C Code vs Assembly

C Code (covers every concept in the course):

```
C
#include <stdio.h>

struct Config {
    int port;
    char *url;
};

int checkAuth(int userID) {
    if (userID == 1337) {
        return 1; // success
    }
    return 0;
}

int main() {
    struct Config cfg = {443, "http://c2.example.com"};
    int i;
    char *commands[3] = {"download", "exec", "exfil"};

    for (i = 0; i < 3; i++) {
        if (checkAuth(1337)) {
            printf("Running %s on port %d\n", commands[i], cfg.port);
        }
    }
    return 0;
}
```

Corresponding Assembly (simplified, as seen in IDA Pro):

```
asm
main:
    push ebp ; function prologue
    mov ebp, esp
    sub esp, 0x20 ; allocate stack for locals

    ; struct Config cfg = {443, "http://c2.example.com"}
    mov dword ptr [ebp-0x10], 443 ; cfg.port = 443
    lea eax, [c2_url] ; load address of string (pointer)
    mov [ebp-0x0C], eax ; cfg.url = pointer

    ; for loop (i = 0; i < 3; i++)
    mov dword ptr [ebp-4], 0 ; i = 0
for_start:
    cmp dword ptr [ebp-4], 3
    jge for_end ; if i >= 3, exit loop

    ; if (checkAuth(1337))
    push 1337
    call checkAuth ; call function
    add esp, 4 ; cdecl: clean stack
    cmp eax, 1
    jne skip ; if not equal (false), skip

    ; printf call (simplified)
    push [ebp-0x10] ; cfg.port
    mov ecx, [ebp-4]
    lea edx, [commands + ecx*4] ; array access with offset
    push dword ptr [edx] ; commands[i]
    call printf
    add esp, 8

skip:
    inc dword ptr [ebp-4] ; i++
    jmp for_start ; loop back

for_end:
    xor eax, eax ; return 0
    mov esp, ebp
    pop ebp
    ret

; checkAuth function
checkAuth:
    push ebp
    mov ebp, esp
    mov eax, [ebp+8] ; userID parameter (from stack)
    cmp eax, 1337
    jne fail
    mov eax, 1 ; return 1
    jmp done

fail:
    xor eax, eax ; return 0
done:
    mov esp, ebp
    pop ebp
    ret
```

Line-by-Line Explanation (how it connects to the course):

- **push ebp; mov ebp, esp; sub esp, 0x20** : Function prologue + stack allocation (functions & stack).
- **mov dword ptr [ebp-0x10], 443** : Writing to structure field using offset (structures).
- **lea eax, [c2_url]** : Loading address into register (pointers).
- **cmp ... jge for_end** : Loop condition (loops).
- **cmp eax, 1; jne skip** : If statement (comparisons & jumps).
- **call checkAuth** : Function call (functions & calling conventions).
- **add esp, 4** : Caller cleaning stack (cdecl convention).
- **mov ecx, [ebp-4]; lea edx, [commands + ecx*4]** : Array access with index (arrays).
- **ret** : Return from function.

Malware analysis tip: Real malware looks similar but with:

- Obfuscated names
- More stack manipulation
- Calls to Windows APIs (CreateProcess, InternetConnect, etc.)
- Heavy use of **CMP + Jcc** for control flow